

Student Score Analyzer

Representative Program Output

text

How many scores will be entered? 0

The number of scores must be at least 1.

How many scores will be entered? 4

Enter score 1 (0-100): 95

Enter score 2 (0-100): 72

Enter score 3 (0-100): 88

Enter score 4 (0-100): 60

Original dynamic array values: 95 72 88 60

Minimum: 60

Maximum: 95

Average: 78.75

Scores greater than average: 2

Pointer arithmetic demonstration: 95 72 88

Vector values using iterators: 95 72 88 60

Enter a target score to search for: 88

88 is present in the vector.

Sorted vector: 60 72 88 95

Vector minimum: 60

Vector maximum: 95

Vector size: 4

Vector capacity: 4

Written Answers

1. What is the difference between the pointer scores and the expression *scores?

scores stores the memory address of the first element in the dynamically allocated array. *scores dereferences that address and produces the integer value stored in the first element. The pointer can be changed to point somewhere else; the dereferenced expression is a score value.

2. Why must memory created with new[] be released with delete[] rather than delete?

new[] creates an array allocation, so delete[] tells C++ to release the complete array allocation correctly. Using delete on memory created with new[] is undefined behavior and can cause incomplete cleanup or heap corruption.

3. What is a memory leak, and how does this program prevent one?

A memory leak occurs when a program loses the pointer to allocated memory without releasing that memory. This program copies the values into the vector, calls delete[] scores exactly once, and then sets scores to nullptr so the pointer no longer refers to released storage.

4. Why is vector generally safer than a manually allocated array?

An STL vector manages its own memory and automatically releases it when the vector leaves scope. It also provides `size()`, checked access through `at()`, and standard algorithms, reducing the risk of leaks, mismatched deallocation, and incorrect size tracking.

5. What can happen to existing vector iterators when an insertion causes reallocation?

When a vector runs out of capacity and reallocates, it moves its elements to a new memory location. Existing iterators, pointers, and references to the old elements are invalidated and must not be used. If no reallocation occurs, insertion still invalidates iterators at or after the insertion position.